# Noise and Speech Activity Analysis

Turning classroom audio into noise and speech context for multimodal fusion

| Assignee | Work Item | Allocated Duration | Status |
|---|---|---|---|
| Humesh Sharma | Story | 1h 30m | In Progress |

## 1. Overview & Scope

This activity analyzes the extracted audio signals to identify **ambient noise levels** and **speech activity patterns**. Audio features such as energy levels, amplitude variation, and short-time signal characteristics were computed for each segment, and used to differentiate between periods of **silence**, **background noise**, and **active speech**.

Noise level trends give insight into classroom environment conditions, while speech activity detection indicates interaction intensity and participation frequency. Results were aggregated over time to produce session-level audio context metrics.

The generated audio insights are structured for fusion with visual engagement data, enabling the multimodal agent to reason about overall session dynamics.

### Analysis Pipeline

**1. Segments**
Windowed audio from the extraction stage
→
**2. Features**
Energy, amplitude variation, short-time characteristics
→
**3. Classify**
Silence, background noise, or active speech
→
**4. Aggregate**
Session-level audio context metrics

## 2. Acoustic Analysis & Feature Specifications

| Analyzed Feature | Classification Target | Functional Role in Session Analysis |
|---|---|---|
| **Energy & Amplitude Variation** | Silence vs. active signal | Computes short-time signal energy across segmented windows to separate quiet intervals from acoustic events. |
| **Ambient Noise Trends** | Background noise floor | Evaluates baseline classroom environment conditions and identifies persistent non-speech acoustic disturbances. |
| **Speech Activity Detection** | Active speech patterns | Measures classroom interaction intensity, vocal engagement, and student/instructor participation frequency. |
| **Temporal Aggregation** | Session-level metrics | Structures time-series audio context insights for downstream fusion with visual engagement indicators. |

Table 1: Acoustic features, what each classifies, and its role in session analysis.

## 3. Client Capture & Streaming

The client component is `frontend/src/components/AudioCapture.jsx`, a React / WebSocket audio client. It captures microphone audio in the browser and streams it to the backend over a WebSocket at `ws://localhost:8000/ws/audio`, where short-time feature extraction produces the noise and speech metrics.

| Element | Role |
|---|---|
| `active` (prop) | Controls capture: when true, capture starts; when false, it stops. |
| `onMetrics` (prop) | Callback accepted by the component for reporting metrics to its parent. |
| `isRecording` (state) | Set to true once the microphone stream has been obtained. |
| `wsRef`, `audioContextRef`, `processorRef`, `inputRef` | Refs holding the WebSocket and the audio-processing objects across renders. |
| `useEffect` on `active` | Starts or stops capture as `active` changes; stops capture on unmount. |
| `startCapture()` | Requests microphone access via `getUserMedia`, marks recording, then opens the WebSocket. |

### Code Skeleton (recovered from the source)

The source listing is scrambled and cut off. The code below is reassembled from its legible fragments; `...` marks content that is missing.

```
// File: frontend/src/components/AudioCapture.jsx   (React / WebSocket Audio Client)
import React, { useEffect, useRef, useState } from 'react';

const AudioCapture = ({ active, onMetrics }) => {
    const [isRecording, setIsRecording] = useState(false);
    const wsRef = useRef(null);
    const audioContextRef = useRef(null);
    const processorRef = useRef(null);
    const inputRef = useRef(null);

    useEffect(() => {
        if (active) {
            startCapture();
        } else {
            stopCapture();
        }
        return () => stopCapture();
    }, [active]);

    const startCapture = async () => {
        try {
            const stream = await navigator.mediaDevices.getUserMedia({ audio: ... });
            setIsRecording(true);

            // Connect WS
            const ws = new WebSocket('ws://localhost:8000/ws/audio');
            wsRef.current = ws;
            ...    // listing ends here in the source
```

## 4. Multimodal Fusion & Context Integration

**Multimodal Fusion Note.** Real-time audio streams captured via the client WebSocket interface (`ws://localhost:8000/ws/audio`) undergo short-time feature extraction to output structured noise and speech activity metrics. These session-level audio insights are fused with visual engagement data so the multimodal agent can reason comprehensively about classroom dynamics.

### Documentation Notes

**Source code partly unreadable.** The code block in the original Activity 3.2 file is scrambled and ends right after the WebSocket is created. The skeleton above is reassembled from legible fragments; the `getUserMedia` options, `stopCapture`, the audio-processing and streaming logic, and any use of `onMetrics` are not visible.

**Analysis code not included.** The feature computation (energy thresholds, noise floor, speech detection) is described in the specification table but no implementation or threshold values appear in the source.

**Hard-coded endpoint.** The WebSocket URL points to localhost, so a deployed setup would need it configurable.